

DOI: 10.13245/j.hust.180305

SM2 签名算法 flush-reload cache 计时攻击

周平^{1,2} 王韬¹ 张帆^{3,4} 赵新杰⁵

(1 军械工程学院信息工程系, 河北 石家庄 050003; 2 69016 部队, 新疆 乌鲁木齐 830049;
3 浙江大学信息与电子工程学院, 浙江 杭州 310027; 4 中国电子科技集团公司保密通信
重点实验室, 四川 成都 610041; 5 北方电子设备研究所, 北京 100083)

摘要 针对现有 flush-reload cache 计时攻击中监测地址选取方法不适用于 SM2 数字签名算法的问题, 提出了一种新的监测地址选取方法, 通过监测含有函数调用指令的 cache 块地址, 利用函数调用导致的 cache 块多次访问特性提高了对 cache 访问行为监测的准确性, 降低了恢复标量 k 的错误率, 实现了对 SM2 数字签名算法的密钥恢复. 实验结果表明: 所提出的方法能够对 SM2 数字签名算法实施有效攻击, 利用一次签名的旁路信息恢复 256 bit 标量 k 的比特错误率仅为 1.09%, 能在 64 次密钥搜索的代价下以 59% 的成功率完全破解签名者私钥.

关键词 公钥密码; 数字签名; 旁路攻击; cache 计时攻击; 椭圆曲线; SM2

中图分类号 TP 393.8 **文献标志码** A **文章编号** 1671-4512(2018)03-0024-06

Flush-reload cache timing attack on SM2 digital signature algorithm

Zhou Ping^{1,2} Wang Tao¹ Zhang Fan^{3,4} Zhao Xinjie⁵

(1 Department of Information Engineering, Ordnance Engineering College, Shijiazhuang 050003, China; 2 PLA Unit 69016, Urumqi 830049, China; 3 College of Information Science & Electronics Engineering, Zhejiang University, Hangzhou 310027, China; 4 Science and Technology on Communication Security Laboratory, China Electronics Technology Group Corporation, Chengdu 610041, China; 5 The Institute of North Electronic Equipment, Beijing 100083, China)

Abstract Previous flush-reload cache attacks cannot be directly adopted to SM2 digital signature algorithm. In this paper, an improved method for selecting the monitored address in flush-reload cache attacks on SM2 was proposed. By taking advantage of multiple cache accesses caused by function callings, the cache lines which contain call instructions were selected to be the monitored addresses in order to increase the accuracy. The experimental results show that the proposed method makes the flush-reload attack on SM2 feasible and effective. The 256 bit scalar k can be recovered with a bit errors rate as only 1.09% by the side channel information of a single signing. The full private key can be recovered with a success rate as 59% through 64 times exhaustive research.

Key words public-key cryptography; digital signature; side-channel attack; cache timing attack; elliptic curve; SM2

cache 计时攻击是一种重要的密码旁路攻击方法, 通常利用一个与密码进程共享 cache 的间谍进程监测密码进程对 cache 的访问信息, 从而获取密钥^[1], 与其他旁路攻击手段相比, 具有无需物理接触密码设备、可通过网络远程获取密钥的优点. cache 计时攻击已被广泛用于破解各类公钥密码

算法^[2-6]和分组密码^[7-9]. 近年来, 随着多核处理器的普及, flush-reload 这一新的 cache 计时攻击方法^[10]实现了对运行于多核处理器上的 RSA 签名算法的密钥恢复. 该方法主要利用间谍进程和密码进程共享内存页面的特性, 间谍进程通过将共享页面中密码进程的特定指令从 cache 架构的所有层级中驱逐

收稿日期 2016-11-16.

作者简介 周平(1988-), 男, 博士; 张帆(通信作者), 副教授, E-mail: fanzhang@zju.edu.cn.

基金项目 国家自然科学基金资助项目(61272491, 61309021, 61472357, 61571063); 中央高校基本科研业务费专项资金资助项目(2015QNA5005); 保密通信重点实验室基金资助项目(9140C110602150C11053).

(flush), 再重载(reload)并计时, 从而可以监测密码进程对指令的访问情况. 随后, Yarom 等^[11]将 flush-reload 攻击应用于 OpenSSL 密码库中 EDCSA 在二元域上的实现上, 恢复标量 k 的比特丢失率达到了 4.26%. Benger 等^[12]将 flush-reload 攻击和格攻击相结合, 以实现 wNAF 算法的 ECC 攻击, 通过采集约 200 次签名可恢复 256 bit 密钥. SM2 椭圆曲线公钥密码算法^[13]是中国国家公钥密码算法标准, 由中国国家密码管理局于 2010 年 12 月发布, 已被广泛用于对基于 RSA 算法的电子认证系统、密钥管理系统、应用系统进行替换和升级改造, 对其安全性进行分析具有重要的研究意义和实用价值. 然而, 当前针对 SM2 安全性的分析相对较少, 对其实现安全性的研究主要集中在功耗攻击^[14]和故障攻击^[15], 在 SM2 cache 攻击方面尚未见相关的报道.

本文从 flush-reload cache 计时攻击的角度, 对运行于多核处理器上的 SM2 数字签名算法的实现安全性进行了研究, 主要针对其中使用最广泛的二进制展开法^[13]. 在 flush-reload cache 计时攻击中, 如何选取有效的 cache 监测地址是最核心问题, 现有的监测地址选取方法主要有两种: 选取每种运算中循环执行的指令地址^[10]; 选取条件分支指令的不同分支地址^[11]. 但在二进制展开法实现的 SM2 中, 现有的两种监测地址选取方式并不适用: 点加运算和倍点运算实现过程中不包含多次循环执行的指令, 无法找到满足第一种方法的监测地址; 二进制展开法实现的标量乘运算的条件分支结构较为简单, 其汇编指令往往只有不到 128 B, 不同分支的跳转指令通常映射在同一个或相邻的 cache 块, 难以对其跳转目标进行区分. 此外, 现有攻击方法^[11]虽然能够获得标量 k 中的绝大多数比特位, 但攻击者并不能确定是哪些比特位置发生了错误, 从而难以直接恢复完整的签名私钥.

针对上述问题, 提出了一种监测地址选取方法, 主要思想是选取 cache 块内包含多个函数调用指令的 cache 块作为监测对象, 利用函数调用造成的 cache 块多次访问以达到提高计时信息采集准确性的目的. 利用该方法, 攻击者能够有效获取 SM2 签名算法计算过程中的标量 k , 从而最终恢复签名私钥. 此外, 由于函数调用的普遍性, 该方法也同样适用于多种其他公钥密码算法的各类软件实现.

1 SM2 flush-reload cache 攻击框架

SM2 数字签名算法 flush-reload cache 攻击框架

如图 1 所示, 图中: Q 为椭圆曲线上一点; G 为基点. 假设攻击者已预先在被攻击服务器中植入间谍程序. 当监测到签名进程启动时, 间谍进程开始启动监测, 监测主要针对 SM2 签名算法标量乘运算步骤, 获取点加运算和倍点运算的执行情况. 攻击者首先分别从点加运算和倍点运算的指令中选取一条指令作为监测对象; 监测过程中间谍进程以一定时间间隔反复将其从 cache 中冲刷和重载并分别记录重载时间. 其中, 监测地址的选取是 flush-reload 攻击的关键, 选取最优的监测地址能使攻击者更准确地监测到签名进程执行的运算, 而不恰当的监测地址选取会导致攻击失败. 签名结束后间谍进程开始对由重载时间构成的旁路信息进行分析, 根据重载时间长短判断签名进程依次对哪种运算的指令进行了访问, 从而得到标量乘运算过程中的点加、倍点运算序列. 由于运算序列与标量 k 的比特值存在对应关系, 因此攻击者可推测出标量 k . 最后, 攻击者根据已知的消息 m 、签名 (r, s) 和推测的标量 k 计算签名私钥 d_A .

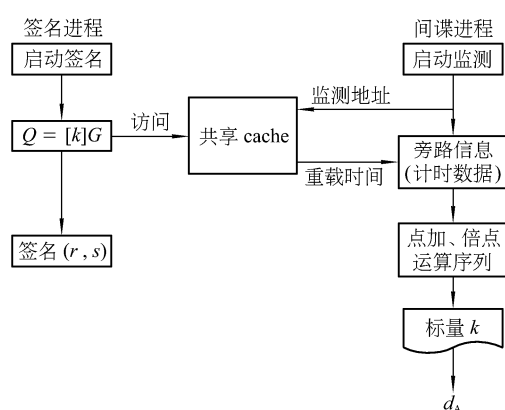


图1 SM2 flush-reload 攻击框架

2 SM2 flush-reload cache 攻击方法

2.1 SM2 实现特性和监测地址选取方法

间谍进程以一个固定的时间间隔冲刷和重载监测地址, 以确定其间密码进程是否执行了相应指令. 由于间谍进程冲刷和重载的行为(其本质也是对监测地址的访问)存在一定概率与密码进程的正常访问发生冲突^[10], 因此不可避免地导致攻击者对密码进程访问监测地址的情况判断错误. 为了尽量减少上述错误, 通常选择监测运算中被频繁循环执行的指令, 这样在一次运算中就能保证总有至少一次访问行为被监测到. 然而在 OpenSSL 以及其他常见的开源密码库中, 实现椭圆曲线上点加运算和倍点

运算的指令通常在一次运算中只是顺序执行一次, 因此执行很容易被遗漏, 而一旦如此, 推测出的点加倍点运算序列就不再正确.

通过研究发现, 监测被频繁循环执行的指令地址其本质是这些地址所对应的 cache 块随着指令的执行而被密码进程多次访问, 从而增加了 cache 访问行为被间谍进程捕获的机会, 降低了发生错误判断的概率. 虽然从高级语言(例如 OpenSSL 使用 C 语言)代码的层面上看, 点加运算和倍点运算中所有的代码均为顺序执行一次, 然而在点加运算和倍点运算的实现过程中会大量调用其他函数, 这使得从汇编代码的层面看, 在实际执行过程中同一个 cache 块中顺序执行的指令会因为函数调用(call 指令)而中断, 调用函数执行完毕后仍然返回该 cache 块并

执行下一条汇编指令.

汇编级的跳转指令造成了 cache 块在事实上以一定时间间隔被多次访问, 这种现象为间谍进程的监测提供了两点便利: 函数调用和返回造成的 cache 块的多次访问增加该访问行为被间谍进程监测到的机会; 调用函数的执行需要一段时间, 增加了被监测 cache 块先后两次访问之间的时间跨度, 减少了对间谍进程监测精度的要求. 因此认为可以将这种具有多个函数调用指令的 cache 块地址作为监测地址. 作为实例, 图 2 给出了 OpenSSL 点加运算中第 673~678 行代码所对应的被监测 cache 块被访问的过程, 该段代码完成了对椭圆曲线参数的初始化工作, 调用了 BN_CTX_get(ctx)函数.

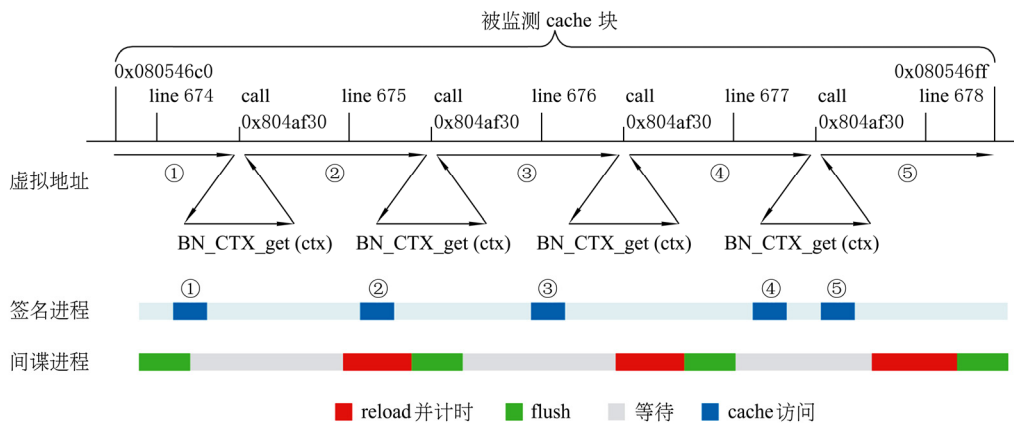


图 2 被监测 cache 块访问过程

当执行 call 指令时程序跳转到调用函数的地址, 在执行完调用函数后返回, 该段代码中的 4 次函数调用指令使得该 cache 块在一次点加运算中被访问 5 次. 密码进程对 cache 的访问行为可能会与间谍进程的 flush 或 reload 行为冲突(①, ②), 或多次访问发生在同一时间间隔内(④, ⑤), 从而导致攻击者不能正确监测到签名进程的指令执行情况; 但由于函数调用行为导致的多次 cache 访问现象增加了访问次数和两次访问间的时间跨度, 从而增加了签名进程 cache 访问行为被正确监测到的机会(③). 须要指出的是, 上述汇编指令结构并非特例, 在不同密码库中的椭圆曲线标量乘运算中不可避免地存在大量函数调用, 而只要存在这样的代码结构, 就意味着可以找到在一次运算中被多次访问的 cache 块.

2.2 SM2 实现特性和监测地址选取方法

利用一个与签名进程并行执行的间谍进程监测签名进程对点加和倍点运算的执行序列. 将上节点加运算和倍点运算中选取的被监测 cache 块的首分别记为 c_a 和 c_d , 当签名开始时, 间谍进程也开始启

动监测并采集由重载时间构成的旁路信息.

a. flush 阶段. 间谍进程(SP)首先使用 cflush 指令分别将 c_a 和 c_d 从 cache 的所有层级中清除.

b. wait 阶段. SP 随后等待一段时间 t_w 不执行任何操作, 根据被监测 cache 相邻两次访问时间间隔将等待时间 t_w 选取为 600 CPU 时钟周期^[4]. 在 SP 等待期间, 签名进程(CP)执行运算并访问相应的监测地址, 若 SP 执行了点加运算, 则 c_a 将被载入 cache; 若 SP 执行了倍点运算, 则 c_d 将被载入 cache.

c. reload 阶段. SP 分别重新访问 c_a 和 c_d , 并分别测量其访问时延 t_A 和 t_D . 若在 wait 阶段 CP 执行了点加运算, 则 c_a 已被载入 cache, SP 重新访问 c_a 所测得的时间为相对较短的 cache 访问时延, 而 SP 重新访问 c_d 所测得的时间为相对较长的内存访问时延; 若在 wait 阶段 CP 执行了倍点运算, 则情况相反.

SP 不断重复上述三个步骤直到签名结束, 在每轮监测中得到一个被监测 cache 块的访问状态矢量 $t_i = [t_{iA}, t_{iD}]^T$, n 轮监测后得到整个签名过程中的被监测 cache 块的访问踪迹 $T = [t_1, t_2, \dots, t_n]$.

对于每一访问状态矢量 t_i 都有一个运算状态 $o_i \in \{F, D, E\}$ 与之对应, 其中 F , D 和 E 分别表示点加运算、倍点运算和其他运算. 令 h 表示区分 cache 访问时延和内存访问时延的阈值, 则运算状态 o_i 与访问状态矢量 t_i 有如下对应关系:

$$o_i = \begin{cases} F & (t_{iA} < h, t_{iD} > h); \\ D & (t_{iA} > h, t_{iD} < h); \\ E & (\text{其他}). \end{cases}$$

去除其他运算状态后可以得到签名算法标量乘运算过程中的点加倍点运算序列, 记为 $O=[o_1, o_2, \dots, o_l]$. 由二进制展开法可知运算序列 O 与标量 $k = \sum_{j=0}^{l-1} k_j 2^j$ 的二进制位取值呈一一对应关系: $k_j=0$

对应一次倍点运算; $k_j=1$ 对应一次倍点运算和一次点加运算, 因此通过运算序列 O 能够完全恢复标量 k . 由于签名 (m, r, s) 已知, 因此一旦标量 k 被攻击者获取, 则可利用签名式 $s=(1+d_A)^{-1}(k-rd_A) \bmod n$ 获得签名者私钥: $d_A=(k-s)(s+r)^{-1} \bmod n$.

3 实验结果与分析

实验在联想 ThinkCentre 台式机上进行, 处理器为 Intel i5-3470, 内存为 4GB DDR3-1600, 操作系统为 Fedora 18. SM2 数字签名算法, 调用 OpenSSL 密码库来实现, 其中标量乘运算使用二进制展开法实现. 实验采用国家商用密码局推荐的素数域 256 bit 曲线参数^[16], 选取的监测地址通过 Eclipse debug 得到.

3.1 cache 计时数据片段分析

分别从点加运算和倍点运算汇编代码中选取发生函数调用次数最多的 cache 块地址作为监测地址进行计时信息采集, 图 3 为一次签名过程中得到的 cache 计时数据片段, 图中: β 为 cache 访问时延, 单位为 CPU 时钟周期; α 为监测轮数. 在图 3 所示的攻击实验中, 根据被监测 cache 相邻两次访问时间间隔将等待间隔设置为 600 CPU 时钟周期; 根据内存和 cache 访问时间差异将阈值设为 180 CPU 时钟周期^[4]. 该图显示了对一次 SM2 签名过程第 20~200 轮监测的情况. 小于阈值的访问时延显示了该时刻加密进程所执行的运算类型. 由于算法运行最初运行时存在不稳定的情况, 第 20~40 轮监测难以准确区分运算类型, 从第 40 轮开始可以准确推测出相应的运算序列为 “... ADADDD DADADDDDDDDDDADDDDDDA...”, 其中: A 表

示点加运算; D 表示倍点运算. 相应的标量 k 的片段为 “...110001100000001000001...”.

实验表明: 利用提出的监测地址选取方法能够采集到有效的计时信息并从中准确地识别恢复出标量 k 的二进制位.

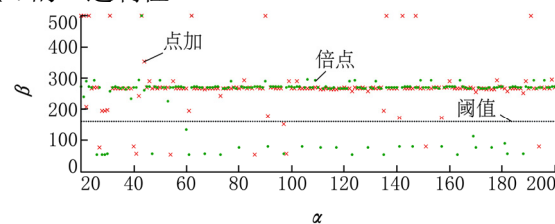


图3 cache 计时数据

3.2 被监测 cache 块访问次数与标量恢复错误比特数关系分析

OpenSSL 中点加运算的汇编指令分布于约 33 个连续的 cache 块, 这里统计了各 cache 在一次运算中实际被访问的次数, 并分别以各 cache 块为监测对象, 恢复出标量 k 的平均错误比特数 A , 如图 4 所示, 图中 B 为一次运算中 cache 块实际被访问的次数. 可见尽管数据有一定的波动性, 但总体上呈现出随着实际被访问次数的增加恢复出标量 k 平均错误比特减少的关系, 两者大体上呈指数关系.

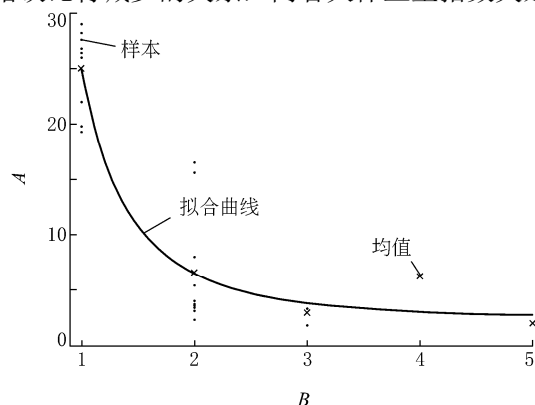


图4 平均错误比特数和被监测 cache 块访问次数的关系

上述统计表明: 函数调用指令能够造成 cache 块在实际运行过程中被多次访问, 且这种结构能够被利用于更加准确地监测标量乘法的运算状态, 在同一 cache 块中调用函数越多, 越能有效提高最终恢复出标量 k 的准确性.

3.3 标量 k 错误比特分析

3.3.1 错误数量统计和位置分布

根据提出的监测地址选取方法, 对 100 次签名过程进行了 cache 计时信息采集并恢复标量 k , 统计恢复出的 256 bit 标量 k 中发生比特翻转、比特丢失、额外比特三种类型错误的平均数量, 如表 1 所示, 总错误数为 2.8 bit, 总错误率为 1.09%.

实验中每个签名的 k 中平均发生 2.8 bit 错误,

错误率为 1.09%，优于文献[11]中平均每个签名发生 25.28 个比特丢失错误数及 4.26% 的比特丢失率。实验对错误比特在标量 k 中发生的位置进行了统计，错误主要集中于前 6 bit，这是由于标量乘运算在计算前若干比特时程序运行尚不稳定所导致。除去前 6 bit 以外，随后的比特位置发生错误的概率呈均匀分布，且每个位置发生错误的概率都小于等于 3%。

表 1 标量 k 平均错误比特数量统计

错误类型	平均错误数/bit	错误率/%
比特翻转	1.16	0.45
比特丢失	0.79	0.31
额外比特	0.85	0.33

3.3.2 错误数量分布

每个签名样本恢复出标量 k 中错误比特数量分布如图 5(a)所示，图中 C 为样本数。可以看出：每个标量 k 中错误比特个数的范围为 0~9 个，其中 25 次攻击直接得到了正确的标量 k ，超过一半以上的攻击中错误比特数小于等于 3。若将每个标量 k 的前 6 bit 忽略，则其错误比特数量分布规律如图 5(b)所示。若忽略前 6 bit，则每个标量 k 中错误比特个数的范围为 0~5 个，其中 59 次攻击在忽略前 6 bit 的情况下被全部正确恢复。其意义在于，尽管在 256 bit 中平均错误比特数只有约 2.8，但攻击者并不能知晓错误比特的位置从而最终获得完整的私钥，但由于有 59% 次攻击能获得完全正确的后 250 bit，且前 6 bit 的变化仅有 $2^6=64$ 种，在如此小的空间内进行穷举搜索完全可行，因此攻击者能够利用本文方法以 59% 的成功率获得一次签名的完整密钥。

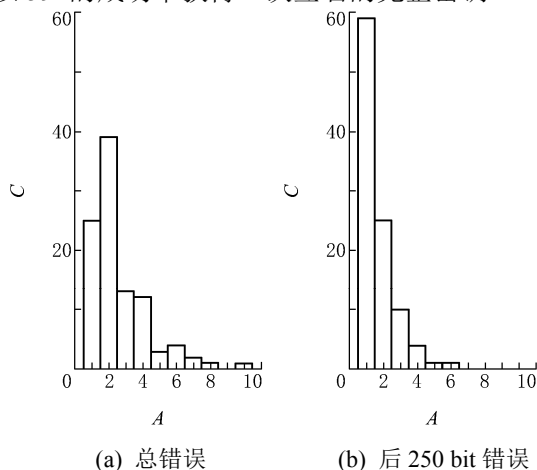


图 5 错误比特数量分布

4 结语

针对现有 flush-reload cache 计时攻击方法不能有效攻破 SM2 数字签名算法的不足，提出了一种新

被监测地址选取方法，由于 cache 块内的函数调用指令能够造成的 cache 块多次访问，因此监测这些 cache 块地址能够起到提高计时信息准确性的目的，从而使得完全获得 SM2 签名私钥成为可能。

本文方法本质上是针对使用二进制展开法实现的标量乘运算步骤，因此也适用于其他标量乘运算通过二进制展开法实现的椭圆曲线类公钥密码。此外，本文方法也可应用于平方乘算法实现的模幂运算。与同样适于远程网络实施密钥分析的简单时间分析相比，本文方法仅需极少的签名样本即可恢复完整密钥。

对 SM2 数字签名算法的攻击实验表明：仅通过单次签名过程的计时信息采集即可准确获得点加倍点运算的序列进而恢复标量 k ；能够使获取 256 bit 标量 k 的错误小于 2.8 bit，错误率小于 1.09%；攻击者能够以 64 次穷举搜索为代价以 59% 的概率完全正确地计算出签名者私钥。可见采用二进制展开法实现的 256 bit SM2 数字签名算法在 flush-reload cache 计时攻击下存在较大的安全风险，在使用时应当考虑增加密钥长度或使用有防护措施标量乘算法。

参 考 文 献

- [1] Page D. Theoretical use of Cache memory as a cryptanalytic side-channel, CSTR-02-003[R]. Bristol: Department of Computer Science, University of Bristol, 2002.
- [2] Acicmez O, Schindler W. A vulnerability in RSA implementations due to instruction cache analysis and its demonstration on openssl[C]// Proc of Topics in Cryptology-CT-RSA. Berlin: Springer, 2008: 256-273.
- [3] Brumley B B, Grabher P. New results on instruction cache attacks[C]// Proc of International Conference on Cryptographic Hardware and Embedded Systems. Santa Barbara: Springer, 2010: 110-124.
- [4] Zhou Ping, Wang Tao, Li Guang, et al. Analysis on the parameter selection method for FLUSH+RELOAD based cache timing attack on RSA[J]. China Communications, 2015, 12(6): 33-45.
- [5] 陈财森, 王韬, 郭世泽, 等. RSA 踪迹驱动指令 cache 计时攻击研究[J]. 软件学报, 2013, 24(7): 1683-1694.
- [6] 陈财森, 王韬, 郭世泽, 等. 针对 RSA 算法的踪迹驱动数据 Cache 计时攻击研究[J]. 计算机学报, 2014, 37(5): 1039-1051.
- [7] 赵新杰, 王韬, 郭世泽, 等. AES 访问驱动 Cache 计时攻击[J]. 软件学报, 2011, 22(3): 572-591.

- [8] 赵新杰, 王韬, 郭世泽, 等. 分组密码 Cache 攻击技术研究[J]. 计算机研究与发展, 2012, 49(3): 453-468.
- [9] 赵新杰, 王韬, 郑媛媛. Camellia 访问驱动 Cache 计时攻击研究[J]. 计算机学报, 2010, 33(7): 1153-1164.
- [10] Yarom Y, Falkner K. Flush+reload: a high resolution, low noise, L3 cache side-channel attack[C]// Proceedings of the 23rd USENIX conference on Security Symposium. San Diego: Association, 2014: 719-732.
- [11] Yarom Y, Benger N. Recovering OpenSSL ECDSA nonces using the FLUSH+RELOAD cache side-channel attack[EB/OL]. [2014-12-20]. <https://eprint.iacr.org/2014/140.pdf>.
- [12] Benger N, van de Pol J, Smart N P, et al. "Ooh Aah... just a little bit": a small amount of side channel can go a long way[C]// Proc of Cryptographic Hardware and Embedded Systems. Berlin: Springer, 2014: 75-92.
- [13] 国家密码管理局. GB/T 32918—2016 SM2 椭圆曲线公钥密码算法[S]. 北京: 国家密码管理局, 2016.
- [14] 史汝辉, 李增局, 杜磊, 等. 一种针对 SM2 解密算法的侧信道攻击方法[J]. 密码学报, 2015, 2(5): 467-476.
- [15] Liu M, Chen J, Li H. Partially known Nonces and fault injection attacks on SM2 signature algorithm[M]. Berlin: Springer International Publishing, 2013.
- [16] 国家密码管理局. GB/T 32918.5—2017 SM2 椭圆曲线公钥密码算法推荐曲线参数[S]. 北京: 国家密码管理局, 2017.

(上接第 17 页)

- organization in small cell networks: a reinforcement learning approach[J]. IEEE Transactions on Wireless Communications, 2013, 21(7): 3202-3212.
- [5] Shi L, Yoo S J. Distributed fair resource allocation for cognitive femtocell networks[J]. Wireless Personal Communications, 2017, 93(4): 883-902.
- [6] Mao T, Feng G, Liang L, et al. Distributed energy-efficient power control for macro-femto networks[J]. IEEE Transactions on Vehicular Technology, 2016, 65(2): 718-731.
- [7] Yan J, Joe I. Energy-efficient resource allocation for heterogeneous cognitive radio network based on two-tier crossover genetic algorithm[J]. Journal of Communications and Networks, 2016, 18(1): 120-122.
- [8] Mokari N, Javan M R, Navaie K. Cross-layer resource allocation in OFDMA systems for heterogeneous traffic with imperfect CSI[J]. IEEE Transactions on Vehicular Technology, 2010, 59(2): 1011-1017.
- [9] Cheung K T K, Yang S, Habzo L. Achieving maximum energy-efficiency in multi-relay OFDMA cellular networks: a fractional programming approach[J]. IEEE Transactions on Communications, 2013, 61(7): 2746-2757.
- [10] Dinkelbach W. On nonlinear fractional programming[J]. Management Science, 1967, 13(7): 492-498.
- [11] Boyd S, Vandenberghe L. Convex optimization[M]. Cambridge: Cambridge University Press, 2004.
- [12] Miao G, Himayat N, Li G Y. Energy-efficient link adaptation in frequency-selective channels[J]. IEEE Transactions on Communications, 2010, 58(2): 545-554.
- [13] Wong C Y, Cheng R S, Lataief K B, et al. Multiuser OFDM with adaptive subcarrier, bit, power allocation[J]. IEEE Journal on Selected Areas in Communications, 1999, 17(10): 1747-1758.
- [14] Chen L, Li X, Ji H. An interference-mitigation channel allocation algorithm for energy-efficient femtocell networks[C]// Proceeding of the 2014 IEEE Wireless Communications and Networking Conference. Istanbul: IEEE, 2014: 2318-2312.